[LOGO AREA]

AMMAR ALZAIEM

CYBERSECURITY | VAPT

SECURITY ASSESSMENT REPORT

Overthewire – 0-9 Levels

REPORT ID	AZ-EH-2026-001
VERSION	1.0
ASSESSMENT DATE	18/10/2026
REPORT DATE	22/10/2026
PREPARED BY	Ammar Alzaiem
CLASSIFICATION	Public

Overview

I worked through the first ten stages to practise using the terminal, navigating the filesystem, searching through files, and processing text. Some levels were straightforward, while others required understanding how Linux handles filenames and command output.

This writeup documents the approach and the commands used to solve each type of challenge.

Level 0 → Level 1: Connecting with SSH

Objective: Connect to the Bandit server and find the password for the next level.

The first step was connecting to the remote machine using SSH. Bandit uses port `2220` instead of the default SSH port, `22`.

```
ssh -p 2220 bandit0@bandit.labs.overthewire.org
```

The starting password is `bandit0`. After logging in, I listed the files in the home directory and read the `readme` file.

```
ls
```

```
cat readme
```

The password displayed by `cat` is used to log in as `bandit1`.

What I learned: SSH provides remote shell access, and `-p` specifies the port. `ls` lists directory contents, while `cat` displays a file's contents.

Level 1 → Level 2: Reading a File Named -

Objective: Find the next password in a file named `-`.

After logging in as `bandit1`, I needed to read a file whose name was just a dash.

```
ls
```

```
cat ./-
```

The problem is that a standalone `-` can be interpreted by some commands as a special argument, often representing standard input or output. Writing `./-` makes it clear that I mean a file in the current directory.

Level 2 → Level 3: Filenames Containing Spaces

Objective: Read a file whose name contains spaces.

The file in this level is named `spaces in this filename`. Since shells normally split command arguments at spaces, typing the filename without quoting it would treat it as several separate arguments.

```
cat "spaces in this filename"
```

Another option is to escape each space with a backslash:

```
cat spaces\ in\ this\ filename
```

Both approaches tell the shell to treat the spaces as part of the filename.

What I learned: Quoting filenames is useful when working with spaces or other characters that have special meaning to the shell.

Level 3 → Level 4: Finding Hidden Files

Objective: Find a hidden file inside the `inhere` directory.

Linux filenames beginning with a dot are treated as hidden by conventional directory-listing tools.

```
ls
```

```
ls -la inhere
```

The `-a` option shows hidden entries, while `-l` displays them in a detailed listing. Once I identified the hidden file, I could read it using `cat` and retrieve the next password.

```
cat inhere/.hidden
```

What I learned: A normal `ls` listing doesn't show hidden files. Using `ls -la` is a useful first step when a directory appears empty or doesn't contain the file I'm looking for.

Level 4 → Level 5: Identifying File Types

Objective: Find the human-readable file among several files in the `inhere` directory.

The directory contains multiple files, and their names don't tell me which one contains readable text. Instead of opening every file individually, I can use the `file` command.

```
file inhere/*
```

This checks the file types and reports what each file appears to contain. I then inspect the file identified as human-readable.

```
cat inhere/-file07
```

The filename above is an example of the naming pattern used in this level; use the filename identified by your own `file` output.

What I learned: File extensions and names aren't always reliable indicators of file content. The `file` command examines file characteristics to help identify their type.

Level 5 → Level 6: Finding a File by Its Properties

Objective: Find a file inside `inhere` with specific properties: human-readable, exactly 1033 bytes, and not executable.

This level is a good example of why `find` is useful. Rather than checking every file manually, I can search by type, size, and permissions.

```
find inhere -type f -size 1033c ! -executable
```

Here's what the options mean:

- `inhere`: Start searching in this directory.
- `-type f`: Match regular files.
- `-size 1033c`: Match files that are exactly 1033 bytes (`c` specifies bytes).
- `! -executable`: Exclude files that are executable.

The command narrows down the results. I then check the matching file's contents and confirm that it contains the next password.

What I learned: `find` becomes much more useful when several conditions are combined. File size, type, and permissions can be used together to narrow down a search.

Level 6 → Level 7: Searching Across the Filesystem

Objective: Find a file somewhere on the system with specific ownership and size requirements.

Unlike the previous level, the target isn't necessarily inside the current user's home directory. The search must start at the filesystem root.

```
find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null
```

The options are:

- `/`: Start at the root of the filesystem.
- `-type f`: Search for regular files.
- `-user bandit7`: Match files owned by `bandit7`.
- `-group bandit6`: Match files belonging to group `bandit6`.
- `-size 33c`: Match files that are 33 bytes.

- `2>/dev/null`: Discard error messages written to standard error, such as permission-denied messages.

After identifying the matching file, I read its contents to obtain the password for the next level.

What I learned: Searching from `/` can find files outside the current directory. Redirecting standard error makes the results easier to read, but it doesn't fix permission problems or make inaccessible files readable.

Level 7 → Level 8: Searching Text with `grep`

Objective: Find the password in `data.txt`, on the line containing the word `millionth`.

The file contains many lines, so reading the entire file isn't necessary. I can use `grep` to find the relevant line.

```
grep "millionth" data.txt
```

`grep` searches text for a specified pattern and prints matching lines. In this case, it finds the line containing `millionth`, alongside the corresponding password.

What I learned: Searching for a known keyword is often faster than manually reading a large file. `grep` is especially useful when analysing logs, configuration files, and command output.

Level 8 → Level 9: Finding the Only Unique Line

Objective: Find the only line in `data.txt` that occurs once.

This file contains many repeated lines. The useful command combines `sort` and `uniq`:

```
sort data.txt | uniq -u
```

`sort` arranges the lines so identical entries appear next to each other. The pipe (`|`) sends the output of `sort` to `uniq`, and `uniq -u` prints lines that occur only once in the sorted input.

Sorting first matters because `uniq` normally compares adjacent lines, not every occurrence throughout an unsorted file.

What I learned: Pipes let me combine small commands into a single workflow. Instead of trying to solve the entire problem with one complicated command, I can use one tool to prepare the data and another to filter it.

Level 9 → Level 10: Extracting Readable Strings

Objective: Find the password inside `data.txt`, which contains mostly non-readable data. The password appears in a human-readable string preceded by several `=` characters.

The `strings` command extracts sequences of printable characters from a file. I can combine it with `grep` to narrow down the results.

```
strings data.txt | grep "="
```

`strings` filters out much of the binary data and displays readable text. Then `grep` searches that output for lines containing an equals sign.

The relevant result contains the password for `bandit10`.

What I learned: Files don't always contain ordinary text, but they may still include readable strings. `strings` is useful for initial inspection of binary files, while `grep` helps filter the output.

Overall Takeaways

Working through these levels gave me practice with several Linux fundamentals:

- Connecting to remote systems using SSH.
- Handling unusual filenames and hidden files.
- Identifying file types with `file`.
- Searching by file properties using `find`.
- Filtering text with `grep`.
- Combining commands with pipes.
- Using `sort` and `uniq` to find unique lines.
- Extracting readable text from files with `strings`.
- Redirecting error output to make command results easier to read.

The biggest lesson is that I don't need to memorise a different command for every problem. Understanding what each command does and how to combine commands is more useful than memorising a list of solutions.